Lecture 16: Python Modules (Chapter 9)

유준수 (Joon Soo Yoo, Ph.D.)

2026년 4월 7일

방산첨단보안연구실 (DAS Lab)
전북대학교 첨단방위산업학과

9.2 Sorting and Reversing Lists

데이터의 질서: 정렬과 반전

정렬(Sorting)이란?

리스트의 요소를 일정한 순서(오름차순 또는 내림차순)로 재배치하는 작업입니다.

- **숫자 데이터:** 크기 순으로 정렬 (`-3, 5, 12...`)
- **문자 데이터:** 사전 순(Dictionary Order)으로 정렬 (`"bat", "cat", "dog"`)

문자열 비교 퀴즈

`"flash"`와 `"flask"` 중 무엇이 더 앞에 올까요?

정답: `"flash"` < `"flask"` (5번째 글자 `'h'`가 `'k'`보다 앞에 있음)

리스트 메서드: `sort()`와 `reverse()`

파이썬 리스트는 자체적으로 순서를 바꾸는 강력한 도구를 가지고 있습니다.

- `list.sort()`: 요소를 **오름차순(Ascending)**으로 정렬합니다.
- `list.reverse()`: 현재 리스트의 순서를 **그대로 뒤집습니다.** (정렬과는 무관하게 단순히 순서만 반전)

코드 예시

```
num_list = [38, 92, 23, 16]


num_list.sort()   # [16, 23, 38, 92]
num_list.reverse()   # [92, 38, 23, 16] (내림차순 효과)
```

주의사항: 대문자와 소문자의 정렬

문자열을 정렬할 때 파이썬은 **ASCII 값**을 기준으로 합니다. 이 때문에 예상치 못한 결과가 나올 수 있습니다.

대문자가 소문자보다 우선!

파이썬에서는 대문자가 소문자보다 작은 값을 가집니다.
따라서 **"Gail"**이 **"apple"**보다 앞에 옵니다. (`'G' < 'a'`)

ASCII 정렬 순서 예시

순서	예시
1순위: 숫자	`"1", "2", "3"`
2순위: 대문자	`"Apple", "Banana", "Zebra"`
3순위: 소문자	`"apple", "banana", "zebra"`

Task: 데이터 정렬 실습 (Sorting & Reversing)

Mission: 보안 로그 ID와 점수를 정렬하세요.

1. `scores = [85, 92, 78, 99, 88]` 리스트를 오름차순으로 정렬한 후 출력하세요.
2. `ids = ["admin", "Guest", "user01", "Root"]` 리스트를 정렬하세요. 대문자가 섞여 있을 때 결과가 어떻게 나오는지 확인해 보세요.
3. `ids` 리스트를 다시 거꾸로(`reverse`) 뒤집어 보세요.

9.3 Common List Operations

리스트 통계와 안전한 복제(Copying)

리스트 통계 함수: max(), min(), sum()

리스트에 담긴 대량의 데이터를 한눈에 요약해주는 강력한 내장 함수들입니다.

- `max(list)`: 가장 큰 요소를 반환 (숫자뿐 아니라 문자열 사전 순서도 가능)
- `min(list)`: 가장 작은 요소를 반환
- `sum(list)`: 모든 요소의 합계 반환 (**숫자 리스트 전용**)

실전 예시: 데이터 분석

```
scores = [88, 92, 75, 99, 82]
print(max(scores))   # 99 (최고점)
print(sum(scores))   # 436 (총점)

cities = ["Seoul", "Jeonju", "Austin", "Chicago"]
print(min(cities))   # "Austin" (사전 순서상 가장 앞)
```

리스트 복제의 함정 (1): 단순 대입 (=)

리스트를 단순히 다른 변수에 대입하는 것은 복사가 아니라 '별명(Alias)'을 붙이는 것과 같습니다.

데이터 공유 (Reference)

```
a = [1, 2, 3]   # 원본 데이터 생성
b = a           # a의 별명 b를 만듦

b[0] = 99       # b를 수정했는데...
print(a)       # 결과: [99, 2, 3] (a도 같이 바뀜!)
```

- **작동 원리:** 변수 a와 b가 메모리 상의 **같은 주소**를 가리킵니다.
- **비유:** 하나의 사물함에 '철수'와 '영희'라는 이름표 두 개를 붙인 것과 같습니다.

리스트 복제의 함정 (2): copy() 메서드

원본 데이터를 안전하게 보존하면서 새로운 리스트를 만들어 작업하고 싶을 때 사용합니다.

데이터 복제 (Shallow Copy)

```
a = [1, 2, 3]    # 원본 데이터 생성
b = a.copy()    # 별개의 복사본 b를 만듦

b[0] = 99      # b를 수정해도...
print(a)      # 결과: [1, 2, 3] (원본이 안전하게 보호됨)
```

- **작동 원리:** 내용은 똑같지만 메모리 상의 **전혀 다른 위치**에 새로운 리스트를 생성합니다.

Task: 원본 데이터를 지키는 지혜 (Wisdom)

Mission: 리스트를 안전하게 복제하고 정렬하세요.

1. `word_list = ["AI", "Defense", "Cyber", "Space"]` 리스트를 만듭니다.
2. 이 리스트를 `wisdom`이라는 이름으로 **안전하게 복사(`copy`)**하세요.
3. `wisdom` 리스트만 알파벳 순으로 정렬(`sort`)합니다.
4. **확인:** 두 리스트를 각각 사용하여 문장을 만들어 출력하고, `word_list`가 변하지 않았는지 확인하세요.

9.5 List Comprehensions

파이썬의 마법: 한 줄로 끝내는 리스트 생성

리스트 컴프리헨션의 구성 요소

기존 리스트나 반복 가능한 객체(Iterable)를 바탕으로 새로운 리스트를 콤팩트하게 만드는 문법입니다.

기본 구조

```
newList = [ expression for loop_variable in iterable ]
```

- **Expression (표현식):** 새로운 리스트에 담길 '결과물'입니다.
- **Loop Variable (반복 변수):** 현재 꺼내온 요소를 지칭하는 변수명입니다.
- **Iterable (반복 대상):** 소스가 되는 기존 리스트나 `range()` 등입니다.

일반 반복문 vs 리스트 컴프리헨션

0부터 9까지의 **제곱수 리스트**를 만드는 두 가지 방법을 비교해 보세요.

[A] 일반 For Loop (4줄)

```
squares = []
for i in range(10):
    squares.append(i * i)
```

[B] 컴프리헨션 (1줄)

```
squares = [i * i for i in range(10)]
```

결과: `[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]`

실전 예시: 문자열 가공하기

컴프리헨션은 문자열 리스트를 다룰 때 더욱 빛을 발합니다.

Dr. Seuss Poem 스타일 리스트 생성

```
words = ["one", "two", "red", "blue"]

# 각 단어 뒤에 " fish"를 붙여 새로운 리스트 생성
poem_lines = [w + " fish" for w in words]

for line in poem_lines:
    print(line)   # 결과: one fish, two fish...
```

데이터 필터링 (Filtering)

`if` 조건문을 추가하면 특정 조건에 맞는 데이터만 골라낼 수 있습니다.

필터링 구조

```
newList = [ expr for var in container if condition ]
```

- **작동 방식:** `if` 조건이 **True**인 경우에만 결과 리스트에 포함됩니다.
- **활용:** 데이터 전처리, 보안 로그 분석, 특정 키워드 추출 등.

필터링 실전 예제: 음수 데이터 선별

리스트에서 특정 조건(음수)에 맞는 데이터만 빠르게 골라내는 방법을 익혀봅시다.

예제: 음수만 골라내기 (Negative Numbers)

```
# 원본 데이터
nums = [21, -1, 50, -9, 300, -50, 2]

# 컴프리헨션 필터링
negatives = [m for m in nums if m < 0]
```

결과 확인 → `[-1, -9, -50]`

Task: 카페 가성비 메뉴 찾기

학생들을 위해 5,000원 이하의 '가성비 메뉴'들만 따로 모아 리스트를 만들려고 합니다.

Mission: 리스트 컴프리헨션으로 메뉴 필터링하기

1. **데이터 준비:** 아래 가격 리스트를 만듭니다.
   `prices = [4500, 5500, 3000, 6000, 4800, 7000, 3500]`
2. **필터링 수행: 리스트 컴프리헨션**과 **if** 조건문을 사용하여 5,000원 이하의 가격만 담은 `cheap_prices` 리스트를 만드세요.
3. **결과 확인:** 생성된 리스트를 출력하여 저렴한 가격들만 잘 모였는지 확인하세요.

실행 결과 예시

```
[4500, 3000, 4800, 3500]
```

팁

대량의 데이터를 처리할 때 `for` 루프와 `append()`를 반복하는 방식 대신 리스트 컴프리헨션을 사용하면 코드의 가독성이 높아지고 실행 속도 측면에서도 유리합니다.

Any Questions?